

Design Pattern

Strategy

Adam HAMOUC

May 2026

1. Intention

Séparer une **famille d'algorithmes** en classes interchangeables, substituables **au runtime** sans modifier le code client.
Signal d'alerte : des **if/switch** sur un type de comportement dans une classe → candidat Strategy.

2. Problème résolu

Toute modification d'un algo impacte toute la classe. Non maintenable, non testable, travail en équipe difficile.

```
void sort(SortType type) {  
    if (type == FRONT_TO_BACK) { /* ... */ }  
    if (type == BY_MATERIAL)    { /* ... */ }  
    // Ajouter un algo => modifier cette classe => risque de regression  
}
```

3. Structure

- **Strategy** : interface commune à tous les algos.
- **Stratégies concrètes** : héritent de Strategy, chacune encapsule un algo.
- **Context** : détient une référence sur la Strategy courante. **Ne choisit pas** — il déclenche. Le **code client** choisit et injecte.

4. Exemple — tri des draw calls

```
// Strategy : interface  
class ISortStrategy {  
public:  
    virtual void sort(std::vector<DrawCall>& dc) = 0;  
    virtual ~ISortStrategy() = default;  
};  
  
// Strategies concretes  
class FrontToBackSort : public ISortStrategy {  
public:  
    void sort(std::vector<DrawCall>& dc) override {  
        std::sort(dc.begin(), dc.end(),  
            [](const DrawCall& a, const DrawCall& b){  
                return a.depth < b.depth;  
            });  
    }  
};
```

```

class MaterialBatchSort : public ISortStrategy {
public:
    void sort(std::vector<DrawCall>& dc) override {
        std::sort(dc.begin(), dc.end(),
            [](const DrawCall& a, const DrawCall& b){
                return a.materialID < b.materialID;
            });
    }
};

// Context : delegate, ne decide pas
class RenderQueue {
public:
    void setStrategy(std::unique_ptr<ISortStrategy> s) {
        strategy = std::move(s);
    }
    void submit(std::vector<DrawCall>& dc) {
        if (strategy) strategy->sort(dc);
        flush(dc);
    }
private:
    std::unique_ptr<ISortStrategy> strategy;
};

// Code client : choisit et injecte la strategie
RenderQueue queue;
queue.setStrategy(std::make_unique<FrontToBackSort>());
queue.submit(opaqueCalls);

// Switch au runtime, RenderQueue inchange
queue.setStrategy(std::make_unique<MaterialBatchSort>());
queue.submit(opaqueCalls);

```

5. Avantages / Inconvénients

Avantages :

- Ajouter un algo = nouvelle classe, sans modifier l'existant (Open/Closed).
- Chaque stratégie testable et développable indépendamment.
- Switch d'algo au runtime sans toucher au Context.

Inconvénients :

- Multiplication des classes si algos nombreux.
- Overhead d'indirection virtuelle — négligeable sauf hot path critique.

6. Cas d'usage engine

- Tri des draw calls (front-to-back, back-to-front, par matériau).
- IA ennemis : attaque, fuite, patrouille — switchable selon l'état.
- Pathfinding : A*, Dijkstra, navmesh — même interface, algos interchangeables.
- Broad phase collision : BVH, grille uniforme, octree.